HACKATHON PROJECT · DEVELOPER GUIDE

LiveCold — Codebase Guide

Everything a new developer needs to understand the codebase and integrate **Pathway** as the real-time streaming backbone.

3 modules · 9 source files · Python 3.10+ · February 2026

✨ v2 — Updated with all MVP improvements

TABLE OF CONTENTS

1. Project Overview
2. Directory Structure
3. Architecture Overview
4. Data Flow
5. Module Reference
    1. sim — IoT Simulators
    2. pipeline — MQTT Listener
    3. decision_engine — Intelligence Layer
6. Shipment State Schema
7. Output Event Schema
8. How to Run Locally
9. Pathway Integration Guide
10. Changelog

1 Project Overview

LiveCold monitors cold-chain logistics in real time. IoT sensors on refrigerated trucks emit temperature and GPS events every few seconds. The system:

- Ingests sensor events via **MQTT**
- Merges GPS + Temperature streams per shipment
- Computes a **risk probability** using a sigmoid-based model
- Makes a **DIVERT / CONTINUE** decision supporting **cost** and **eco** modes
- Tracks system-wide metrics including **cargo value saved** by diversions

> 🎯 **Pathway** will replace `pipeline/livecold_pipeline.py` with a production-grade streaming engine. The entire `decision_engine/` stays untouched.

2 Directory Structure

```
livecold/
├── sim/
│   ├── config.py              # Shared constants: broker, topics, cities
│   ├── shipment_factory.py    # Generates 25 randomized shipment objects
│   ├── gps_simulator.py       # Publishes GPS events to MQTT
│   └── temp_simulator.py      # ✦ v2: drift cap + auto-recovery
│
├── pipeline/
│   └── livecold_pipeline.py   # ← TARGET FOR PATHWAY · ✦ v2: logging + decay + DIVERT alerts
│
├── decision_engine/
│   ├── evaluator.py           # Main entry point: evaluate_shipment()
│   ├── risk_model.py          # Sigmoid risk model
│   ├── diversion_optimizer.py # ✦ v2: constants + eco mode
│   └── metrics_engine.py      # ✦ v2: cargo_value_saved_inr
└── venv/
```

3 Architecture Overview

```
sim/shipment_factory.py --generates--> 25 Shipment Objects | +-- gps_simulator.py --
MQTT(livecold/gps)--> MQTT Broker :1883 | | +-- temp_simulator.py --MQTT(livecold/temp)--> MQTT
Broker :1883 | pipeline/livecold_pipeline.py logging · state merge · sync gate |
evaluate_shipment(state) _______________|_____________ | | | risk_model diversion_optimizer
metrics_engine compute_risk compute_diversion MetricsEngine.update
|_______________|_____________| | | log output + DIVERT alerts + JSON
```

4 Data Flow

Step 1 — Shipment Generation

`generate_shipments()` creates 25 shipments once (cached, `random.seed(42)`). Each has origin/destination, speed, cargo type, safe temp range, and `temp_mode`.

Step 2 — Sensor Publishing

Simulator	MQTT Topic	Key Fields
`gps_simulator.py`	`livecold/gps`	`lat`, `lon`, `eta_minutes_remaining`
`temp_simulator.py`	`livecold/temp`	`temp_c`, `safe_min/max_temp`, `cargo_value_inr`

Step 3 — Pipeline Ingestion (v2)

1. Parse JSON payload
2. Initialise `shipment_state` dict for new IDs
3. Update `temp` or GPS fields by topic
4. Track `exposure_minutes` with **gradual decay** (not hard reset)
5. Sync gate → `process_shipment()`

Step 4 — Intelligence & Output

`evaluate_shipment()` → risk model → diversion optimizer (cost/eco) → metrics update → returns JSON. DIVERT events trigger 🚨 log warning.

---

5 Module Reference

5.1 — 📁 `sim/`

config.py

Constant	Value	Purpose
`BROKER`	`"localhost"`	MQTT broker host
`PORT`	`1883`	MQTT port
`GPS_TOPIC`	`"livecold/gps"`	GPS publish topic
`TEMP_TOPIC`	`"livecold/temp"`	Temp publish topic
`SIMULATION_INTERVAL`	`2`	Tick interval (seconds)
`NUMBER_OF_SHIPMENTS`	`25`	Trucks to simulate

shipment_factory.py — Product Profiles

Product	Safe Min °C	Safe Max °C	Value (INR)	Sensitivity
Vaccines	2	8	25,00,000	HIGH
Frozen Meat	-20	-10	15,00,000	MEDIUM
Dairy	2	6	8,00,000	MEDIUM
Seafood	0	4	12,00,000	HIGH

temp_simulator.py — Temperature Modes V2 UPDATED

Mode	Behaviour
`"stable"`	±0.5°C noise — stays within safe range
`"drift"`	Creeps up 0.05/tick (HIGH) or 0.02/tick (MEDIUM). **v2:** auto-recovers when >safe_max + 6°C — prints `🔧 Refrigeration repaired` and switches to stable

---

5.2 — 📁 `pipeline/` V2 UPDATED

> ⚠️ **Primary file to replace for Pathway integration.**

Function	Role
`start_pipeline()`	Connects MQTT, subscribes, starts loop. Logs startup via `log.info()`
`on_message()`	Parses payload, merges state, applies **exposure decay**, calls process_shipment
`process_shipment()`	Sync gate + evaluate + **DIVERT alert** with net saving + metrics box every 25 events

Key constant

```
EXPOSURE_DECAY_PER_TICK = 0.5  # decay rate when temp returns to safe range
```

v2 log output

```
18:42:01 [INFO]    🚀 LiveCold Pipeline Started
18:42:03 [INFO]    ✅ CONTINUE | SHP_3 | temp=5.1°C | risk=5%
18:42:05 [WARNING] 🚨 DIVERT   | SHP_5 | temp=11.2°C | risk=83% | saving ₹18,500
          ╟── SYSTEM METRICS (every 25 events) ──╢
            Cargo saved  : ₹1,85,000  ← 💰 value our system protected
            Diversions   : 3 (12.0%)
          ╚═══════════════════════════════════╝
```

5.3 — 📁 `decision_engine/`

> ✅ **Pure Python — no MQTT or I/O.** Works unchanged with Pathway. Wrap `evaluate_shipment` in `@pw.udf`.

evaluator.py

Step	Call	Effect
1	`compute_risk(state)`	Adds `risk_probability` to state
2	`compute_diversion_decision(state)`	Returns action + costs dict
3	`metrics_engine.update()`	Updates all counters
4	Build output	Returns structured JSON

risk_model.py

```
deviation  = |temp - nearest safe boundary|  (0 if in range)
base_score = deviation x 1.8 + (exposure_minutes x 0.2)
risk_p     = sigmoid(base_score x min(eta_minutes/300, 1.5))
           = fixed 0.05 if temp within safe range

<0.3 → LOW  |  <0.6 → MEDIUM  |  ≥0.6 → HIGH
```

diversion_optimizer.py V2 UPDATED

Named constants (v2 — easy to tune):

```
FUEL_COST_PER_KM          = 40    # ₹/km
EMISSION_FACTOR           = 0.25  # kg CO₂/km
RISK_REDUCTION_IF_DIVERT  = 0.7   # 70% risk reduction when diverted
DIVERT_RISK_THRESHOLD     = 0.6   # minimum risk to consider diversion
ECO_CO2_PENALTY_THRESHOLD = 20    # kg CO₂ above which eco mode avoids diversion
```

Decision logic (v2 — supports eco mode):

```
if risk_p < 0.6  →  CONTINUE  "Risk below threshold"

elif objective_mode == "cost":
    if total_divert_cost < expected_loss  →  DIVERT  "Cost optimal under high risk"
    else                                  →  CONTINUE "Diversion not economical"

elif objective_mode == "eco":
    if co2_delta > 20 AND saving is low  →  CONTINUE "Avoid CO₂"
    elif total_divert_cost < expected_loss →  DIVERT  "Eco: cost optimal + CO₂ OK"
    else                                  →  CONTINUE "Not economical (eco)"
```

> 💡 To activate eco mode for a shipment, set `"objective_mode": "eco"` in the state initialisation block inside `livecold_pipeline.py`.

metrics_engine.py V2 UPDATED

Field	Description
`total_events_processed`	Total update() calls
`total_value_monitored_inr`	Sum of cargo values processed
`total_expected_loss_inr`	Cumulative expected loss
`total_high_risk_events`	Events with risk > 0.5
`total_diversions`	DIVERT actions taken
`diversion_rate_percent`	diversions / events × 100
`total_co2_delta_kg`	Total extra CO_2 from diversions
`cargo_value_saved_inr`	**NEW v2 — net money saved: loss_prevented - diversion_cost**

6 Shipment State Schema

```
{
  "shipment_id": "SHP_1",       # identity
  "product_type": "Vaccines",
  "lat": 13.0827,               # from GPS events
  "lon": 80.2707,
  "temp": 9.4,                  # from temp events
  "last_temp": 8.9,
  "safe_min_temp": 2,
  "safe_max_temp": 8,
  "value_inr": 2500000,
  "nearest_hub_distance_km": 45,
  "eta_minutes_remaining": 180,
  "objective_mode": "cost",     # "cost" | "eco"  ← change here for eco mode
  "exposure_minutes": 3.5,      # v2: float — decays by 0.5/tick when in range
  "risk_probability": 0.743     # set by evaluator
}
```

7 Output Event Schema

```
{
  "shipment_id": "SHP_1",
  "lat": 13.0827, "lon": 80.2707, "temp": 9.4,
  "risk_probability": 0.743,
  "recommended_action": "DIVERT",
  "expected_loss_inr": 185000.0,
  "diversion_cost_inr": 1800.0,
  "co2_delta_kg": 11.25,
  "decision_reason": "Cost optimal under high risk",
  "objective_mode": "cost",      # new v2 field
  "timestamp": "2025-01-01T12:00:00.000"
}
```

8 How to Run Locally

Prerequisites: Python 3.10+, Mosquitto broker on `localhost:1883`, `pip install paho-mqtt`

```
# Terminal 1
cd livecold
python pipeline/livecold_pipeline.py

# Terminal 2
cd livecold/sim
python gps_simulator.py

# Terminal 3
cd livecold/sim
python temp_simulator.py
```

9 Pathway Integration Guide

What Pathway Replaces

```
CURRENT: paho-mqtt → on_message() → shipment_state dict → evaluate_shipment() PATHWAY:
pw.io.mqtt.read() → pw.Table → stateful UDF → evaluate_shipment()
```

Files: Modify vs Keep

File	Action
`pipeline/livecold_pipeline.py`	🔴 **Replace entirely**
`decision_engine/evaluator.py`	🟢 Leave — wrap in `@pw.udf`
`decision_engine/risk_model.py`	🟢 Leave as-is
`decision_engine/diversion_optimizer.py`	🟢 Leave — constants already externalised
`decision_engine/metrics_engine.py`	🟡 Optional — replace with `pw.reducers`
`sim/`	🟢 Leave — still publish to MQTT

1. Read from MQTT

```
import pathway as pw
from decision_engine.evaluator import evaluate_shipment

gps_stream  = pw.io.mqtt.read("livecold/gps",  broker="localhost", port=1883)
temp_stream = pw.io.mqtt.read("livecold/temp", broker="localhost", port=1883)
```

2. Join on shipment_id

```
joined = gps_stream.join(temp_stream, pw.left.shipment_id == pw.right.shipment_id)
```

3. Exposure decay — stateful transformer (v2)

```
EXPOSURE_DECAY_PER_TICK = 0.5   # must match pipeline constant

@pw.stateful_transformer
class ExposureTracker:
    def __init__(self): self.exposure = 0.0

    def process(self, temp, safe_min, safe_max):
        if temp < safe_min or temp > safe_max:
            self.exposure += 1
        else:
            self.exposure = max(0.0, self.exposure - EXPOSURE_DECAY_PER_TICK)
        return self.exposure
```

4. Apply evaluate_shipment as UDF

```
@pw.udf
def run_decision_engine(state: dict) -> dict:
    return evaluate_shipment(state)

output = joined.select(result=run_decision_engine(pw.this.merged_state))
```

5. Replace MetricsEngine with pw.reducers (includes cargo_value_saved)

```
metrics = output.reduce(
    total_events=pw.reducers.count(),
    total_diversions=pw.reducers.sum(pw.this.recommended_action == "DIVERT"),
    cargo_saved=pw.reducers.sum(
        pw.apply(
            lambda r: max(0, r["expected_loss_inr"] - r["diversion_cost_inr"])
                      if r["recommended_action"] == "DIVERT" else 0,
            pw.this.result
        )
    ),
)
```

6. Sink & Run

```
pw.io.jsonlines.write(output, "./decisions_output.jsonl")
pw.run()
```

10 Changelog

v2	**diversion_optimizer:** Named constants · Eco mode with CO_2 branching **metrics_engine:** `cargo_value_saved_inr` stat **temp_simulator:** Drift mode ceiling + auto-recovery (🔧 repaired event) **pipeline:** `logging` replaces `print()` · Exposure decay (0.5/tick) · 🚨 DIVERT alert formatting
v1	Initial release